Beyond Destinations: Instruction-Aware Graph Path Planning and Navigation with OpenStreetMap

Anonymous Authors

***Abstract*— We propose BD-Nav, a novel instruction-aware navigation framework that integrates large language models with OpenStreetMap graphs to generate routes that align with user intent. Unlike prior approaches that only extract destinations from language, BD-Nav interprets free-form instructions to identify not only subgoals but also intermediate routes that satisfy conditions such as safety, weather, or avoidance constraints. Our system employs sequential large language model modules for intent parsing, weight function generation, and path evaluation, enabling iterative refinement of graph-based routes. We validate BD-Nav on real-world outdoor experiments with a quadruped robot, demonstrating its ability to follow user instructions without pre-built maps. Results show improved success rates in multi-subgoal scenarios and enhanced robustness in incomplete OpenStreetMap regions compared to ablation variants. This work highlights the potential of combining open-source maps and large language model reasoning to achieve practical, user-centered robot navigation.**

I. Introduction

As autonomous driving technology advances, robots with autonomous navigation capabilities are becoming increasingly common in everyday life. Representative examples include cleaning robots, guide robots, and serving robots [1]. For these robots to effectively operate alongside humans, they must be able to understand natural language commands and navigate accordingly [2]. Traditionally, robots have received navigation commands, such as destinations, through physical interfaces including buttons [3] or mobile applications [4]. While these methods work well in environments where spatial information is fully known, they face limitations when robots encounter unfamiliar places or outdoor areas where complete maps are unavailable. In such cases, it becomes necessary for robots to interpret free-form natural language instructions and plan navigation paths that align with the user's intent [5]. These instructions may include explicit information such as destinations and directions. They may also contain implicit details, such as preferred routes, current environmental conditions, and constraints related to the objects being delivered [6]. Users often expect more than just the shortest path as they may require routes that consider safety, weather, or delivery priorities.

Existing research on language-based spatial understanding and navigation [7]–[14] can be broadly categorized by their focus. Previous studies, such as [7], [8], focus primarily on place recognition using language. While this provides semantic context to assist navigation, it alone cannot generate routes or control robot movement. Other works, such as [9], combine simple natural language commands including narrow path and wide path with occupancy grid maps to create costmaps for path planning. However, these methods are limited in linguistic expressiveness and lack real-world navigation experiments for validation.

Several recent approaches directly tackle natural language-based navigation. WanderGuide [11] uses a large model to extract destination from user instructions, then matches it to a pre-built database of landmarks containing text labels and images. While effective for identifying destinations, it requires database preparation in advance and only supports one destination per instruction. E2Map [12] extends this idea by using visual-language maps (VLMaps) [15] to navigate sequentially to multiple destinations. However, this method still requires a pre-generated visual-language map and relies on users to specify exact landmark names, limiting its flexibility in real-world settings.

LM-Nav [13] leverages foundation models, including large language models (LLMs) and visual-language models (VLMs), to parse object labels from instructions and build a topological graph for navigation. Although this enables a richer multi-modal understanding, it relies on pre-collected image data to verify destination arrival and evaluates success solely by whether the robot reaches the final destinations, ignoring the quality of the overall route. Similarly, OpenBench [14] integrates language instructions with OpenStreetMap (OSM) [16] to support multi-destination navigation in outdoor last-mile delivery tasks. While OSM removes the need for manual map preparation and increases environmental scalability, this method still focuses only on reaching destinations rather than ensuring the route reflects the user's specific preferences or constraints.

To address these limitations, we propose BD-Nav, a framework that leverages LLMs to interpret user instructions and cross-reference them with OSM data to determine both desired destinations and corresponding routes. Unlike prior methods that only identify destinations, BD-Nav considers the nuances in user instructions when selecting navigation routes, ensuring paths align closely with the user's intent. In addition, BD-Nav presents the generated route to the user for confirmation and allows iterative refinement, ensuring that the final plan accurately represents user expectations. Finally, the confirmed route is integrated into a rule-based quadruped robot navigation framework, enabling stable and reliable real-world navigation to multiple destinations.

The contributions of BD-Nav are summarized as follows:

- We propose a method to generate graph paths that reflect the user's intent by leveraging an LLM to interpret and compare user instructions with OSM graphs, enabling the identification of not only subgoals but also the intermediate routes connecting them.

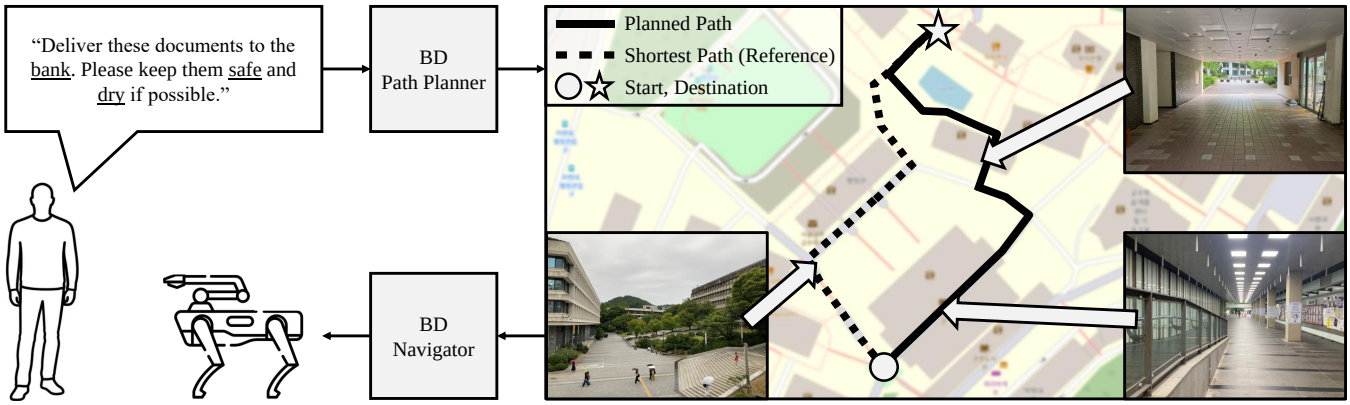


Fig. 1. BD-Nav concept scenario. The robot navigates primarily through buildings to deliver documents to the destination while avoiding the rain, following the user's instructions.

- We introduce a path evaluator module that validates the generated route and improves it when necessary, producing a navigation path that better matches the user's desired behavior.
- We integrate the generated graph paths into a practical navigation framework and validate the approach through real-world experiments with quadruped robots, demonstrating that BD-Nav can be effectively applied to real-world navigation scenarios.

II. Related Works

A. Pose-Goal Navigation

Traditional pose-goal-based navigation studies [17], [18] deliver a numeric goal pose to the robot, generate a path on a pre-built occupancy grid map, and navigate to the destination. Since these methods directly use accurate goal poses, there is little chance that the destination will be misinterpreted by the robot, and the robot can immediately begin navigation based on the provided pose goal. Such pose-goal-based navigation performs well in environments where the robot has prior knowledge of the area and the exact goal pose is known. However, the requirement for precise goal poses also limits the applicability of these methods to diverse and unstructured environments, making them less flexible in real-world scenarios.

B. Landmark-based Navigation

Landmark-based navigation studies [11], [19], [20] represent destinations using words or images, allowing only a single destination to be provided at a time. ViNG [19], for instance, pre-generates a topological graph based on images. When a goal image and the current image are given, the system searches the graph for the most similar nodes and generates a path by connecting these nodes. The destination is set as a single node, and the path to that node is selected as the shortest path using Dijkstra's algorithm [21] within the graph coordinates. DRAGON [20] first performs mapping of the environment using 2D Simultaneous Localization and Mapping (SLAM). During this process, the user can arbitrarily designate landmarks, where front-view images and positional information are stored in a database. At run time, the user specifies the desired goal through text input, and the system leverages the CLIP model [22] to compute text–image similarity in a multi-modal fashion. The most similar goal landmark is selected as the destination, and path planning is carried out using the A* algorithm [23]. WanderGuide [11] creates an occupancy grid map in real time through SLAM while simultaneously storing text labels and images for landmarks. The system then compares the similarity between the user-provided text-based goal and the stored landmarks to find the most relevant goal, and generates a navigation path accordingly. Delivering abstracted goal information, such as text or images, is more convenient for the user compared to specifying exact numeric goal poses. However, in practice, users may want the robot to visit multiple destinations sequentially. Additionally, when moving between destinations, users may desire the robot to follow routes that satisfy specific conditions rather than simply taking the shortest path. To meet these requirements, there is a need for algorithms capable of generating paths that include multiple destinations and consider user-specified conditions for intermediate routes.

TABLE I

Existing studies on language-based navigation.

	Multiple Goals	w/o Prior Map	Intermediate Route
[12], [13]	✓		
[14]	✓	✓	
Ours	✓	✓	✓

C. Language-based Navigation

Language-based navigation studies [12]–[14] allow the user to deliver navigation commands as natural language sentences. The robot then parses necessary information, such as destination names, from the sentences and uses this information to select subgoals for navigation. Since multiple destinations can be included within a single sentence, these methods can support navigation that sequentially visits multiple destinations. Furthermore, additional information such as directions or distances can be included, improving the accuracy of destination selection. However, similar to the landmark-based approaches, language-based methods only

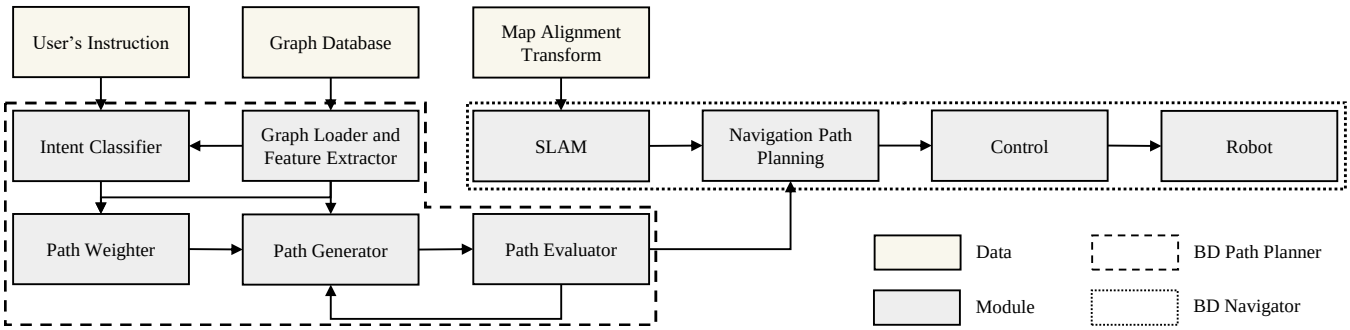


Fig. 2. System architecture. The system generates an OSM-based graph path from the user's instruction and uses a rule-based navigation framework to follow the generated path.

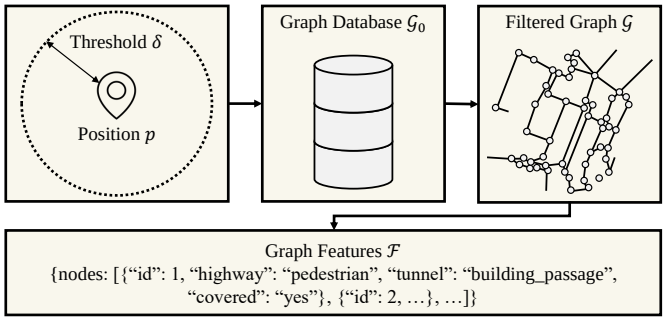


Fig. 3. Process of the graph loader and feature extractor. A filtered graph is loaded from the OSM graph database based on the initial position and distance threshold. The labels and feature information of nodes and edges within the graph are then extracted to generate graph features.

focus on selecting destinations, without considering the intermediate routes between them. When a user wants to control not only the destinations but also the overall path taken by the robot throughout the entire journey, these methods are insufficient.

To the best of our knowledge, BD-Nav is the first approach capable of receiving sentence-level instructions from the user for robot navigation, which include not only destinations but also route conditions and contextual information. Through this capability, BD-Nav can reflect the user's intent not only in the destinations but also in the paths connecting them. Moreover, the generated routes can be directly utilized for robot navigation, enabling the robot to perform navigation that aligns with the user's instructions.

III. Methods

A. Problem Definition

The goal of this paper is to enable a quadruped robot to autonomously navigate to out-of-sight destinations in outdoor environments based on natural language user instructions. To generate a graph path, the OSM database is used to load a graph representing the surrounding environment. As shown in Fig. 2, the BD Path Planner generates an OSM-based graph path by processing the user instruction together with the loaded OSM database. Subsequently, the BD Navigator applies SLAM, navigation path planning, and control algorithms to the generated graph path to produce control inputs for the actual robot. These control inputs are then sent to the robot to execute autonomous navigation. This problem can be mathematically formulated as:

$$P = \pi(T, \mathcal{G}_0), \tag{1}$$

$$U = \upsilon(P, S). \tag{2}$$

where T denotes the user-provided text instruction, $\mathcal{G}_0$ represents the OSM graph database, and P is the generated graph path. The graph path planning function π defines the mapping from T and $\mathcal{G}_0$ to P. S represents the sensor data acquired from sources such as LiDAR and robot motion sensors. U denotes the control input to be applied to the robot, and υ is the navigation function that generates U by utilizing both P and S.

B. Graph Loader and Feature Extractor

Based on the robot's current position, the relevant portion of the OSM graph is loaded for navigation. From the loaded graph, node and edge features are extracted for use by subsequent modules such as the intent classifier and path weighter, as shown in Fig. 3. This process is formulated as:

$$\mathcal{G} = (\mathcal{V}, \mathcal{E}) = g(\mathcal{G}_0, p, \delta), \tag{3}$$

$$\mathcal{F} = \phi(\mathcal{G}), \tag{4}$$

where p denotes the current robot position and δ is a predefined distance threshold for graph loading. $\mathcal{G}$ is the OSM subgraph loaded from the global database $\mathcal{G}_0$. The vertex set is $\mathcal{V}$ and the edge set is $\mathcal{E}$. The function g is the graph loader that returns $\mathcal{G}$ from the global OSM database $\mathcal{G}_0$ given p and δ. $\mathcal{F}$ denotes the set of extracted features, and ϕ is the feature-extraction function applied to $\mathcal{G}$.

C. Intent Classifier

The intent classifier module shown in Fig. 4 takes the user instruction and the graph features as inputs, leveraging a LLM to interpret the user's intent. The system prompt is designed to extract four types of information from the instruction: destinations, waypoints, avoidance constraints, and route constraints. The output can be expressed as:

$$\{\mathcal{D}, \mathcal{W}, \mathcal{A}, \mathcal{C}\} = \eta(T, \mathcal{F}), \tag{5}$$

where $\mathcal{D}$, $\mathcal{W}$, $\mathcal{A}$, and $\mathcal{C}$ denote the destination set, the ordered waypoint set, the avoidance constraints, and the route constraints. The function η parses the instruction T together with the features $\mathcal{F}$ to extract these elements. The sets $\mathcal{W}$, $\mathcal{A}$, and $\mathcal{C}$ are forwarded to the path weighter, while $\mathcal{D}$ is sent to the path generator.

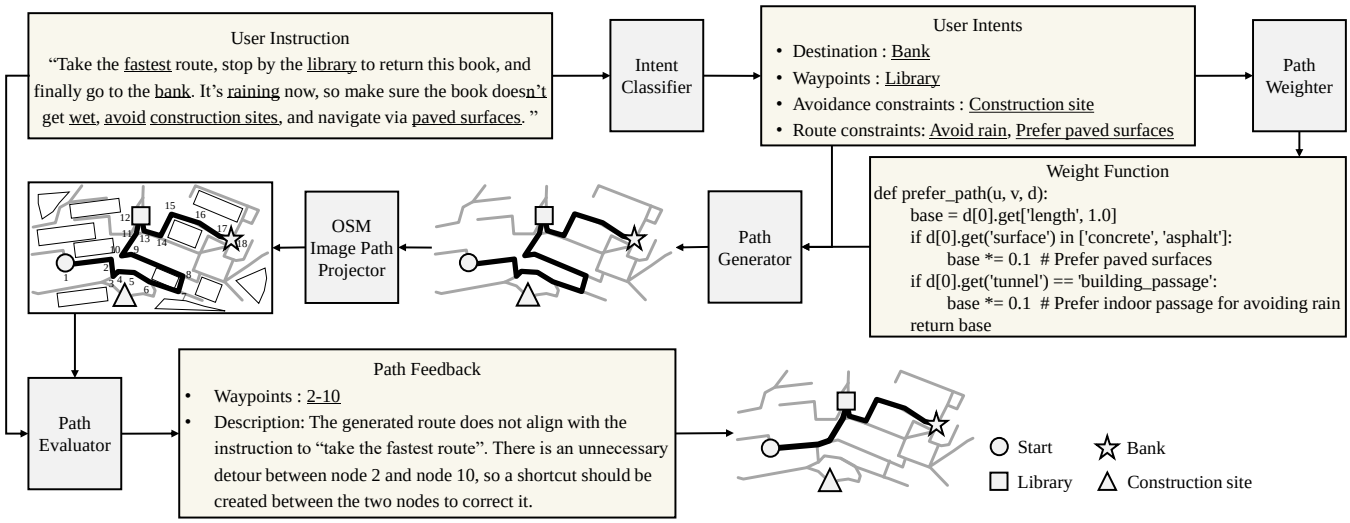


Fig. 4. Path planning process from the intent classifier to the path evaluator. The system receives a user instruction as input, identifies destination and waypoint navigation intents, and forwards them to the path weighter module, which generates weight function code for OSM graph path planning.

D. Path Weighter

The path weighter constructs a text-derived weight function that encodes the user's preferences and constraints. It combines extracted graph features with route constraints inferred from the user's intent and passes the resulting function to the path generator. The function assigns a nonnegative cost to every edge in the graph:

$$\Psi = \psi(\mathcal{F}, \mathcal{C}), \quad \Psi : \mathcal{E} \to \mathbb{R}_{\geq 0}, \tag{6}$$

where Ψ represents the weight function, and ψ denotes the process of generating Ψ using the graph features $\mathcal{F}$ and the route constraints $\mathcal{C}$.

E. Path Generator

Using the loaded OSM graph, destinations, waypoints, avoidance constraints, starting point, and the weight function for generating a path, the path generator performs a graph search using the A* algorithm [23] to compute a feasible path that satisfies the user-defined conditions. The generated path is then forwarded to the path evaluator:

$$P = \xi(\mathcal{G}, \mathcal{D}, \mathcal{W}, \mathcal{A}, p, \Psi), \tag{7}$$

$$P = \underset{Q \in \mathcal{F}(\mathcal{G}, p, \mathcal{D}, \mathcal{W}, \mathcal{A})}{\arg\min} \sum_{e \in Q} \Psi(e). \tag{8}$$

where ξ denotes the graph-search operator and $\mathcal{F}(\mathcal{G}, p, \mathcal{D}, \mathcal{W}, \mathcal{A})$ is the set of admissible paths in $\mathcal{G}$ that start at p, reach a destination in $\mathcal{D}$, visit the waypoints $\mathcal{W}$ in order, and avoid the constraints encoded by $\mathcal{A}$. A candidate Q denotes a path in $\mathcal{G}$ represented as an ordered sequence of edges $Q = (e_1, \ldots, e_K)$ with each $e_i \in \mathcal{E}$. The sum $\sum_{e \in Q} \Psi(e)$ accumulates the edge costs along Q and e ranges over the edges of Q.

F. Path Evaluator

The path evaluator is a VLM-based module that determines whether the generated OSM graph path aligns with the user's intent. It takes the user instruction and a visualization of the graph path overlaid on the OSM map as input. This image-based representation provides spatial context, which is difficult to fully capture from graph data alone. While positional and geometric features could be fed into an LLM as text, prior work [24] shows that LLMs struggle to accurately interpret spatial relationships. Thus, a VLM-based approach is adopted for higher accuracy. If the path is valid, it is passed to the navigation module for execution. If unsuitable, the evaluator generates feedback, including new destinations or missing roads, and sends it back to the path generator for refinement. This process is mathematically formulated as follows:

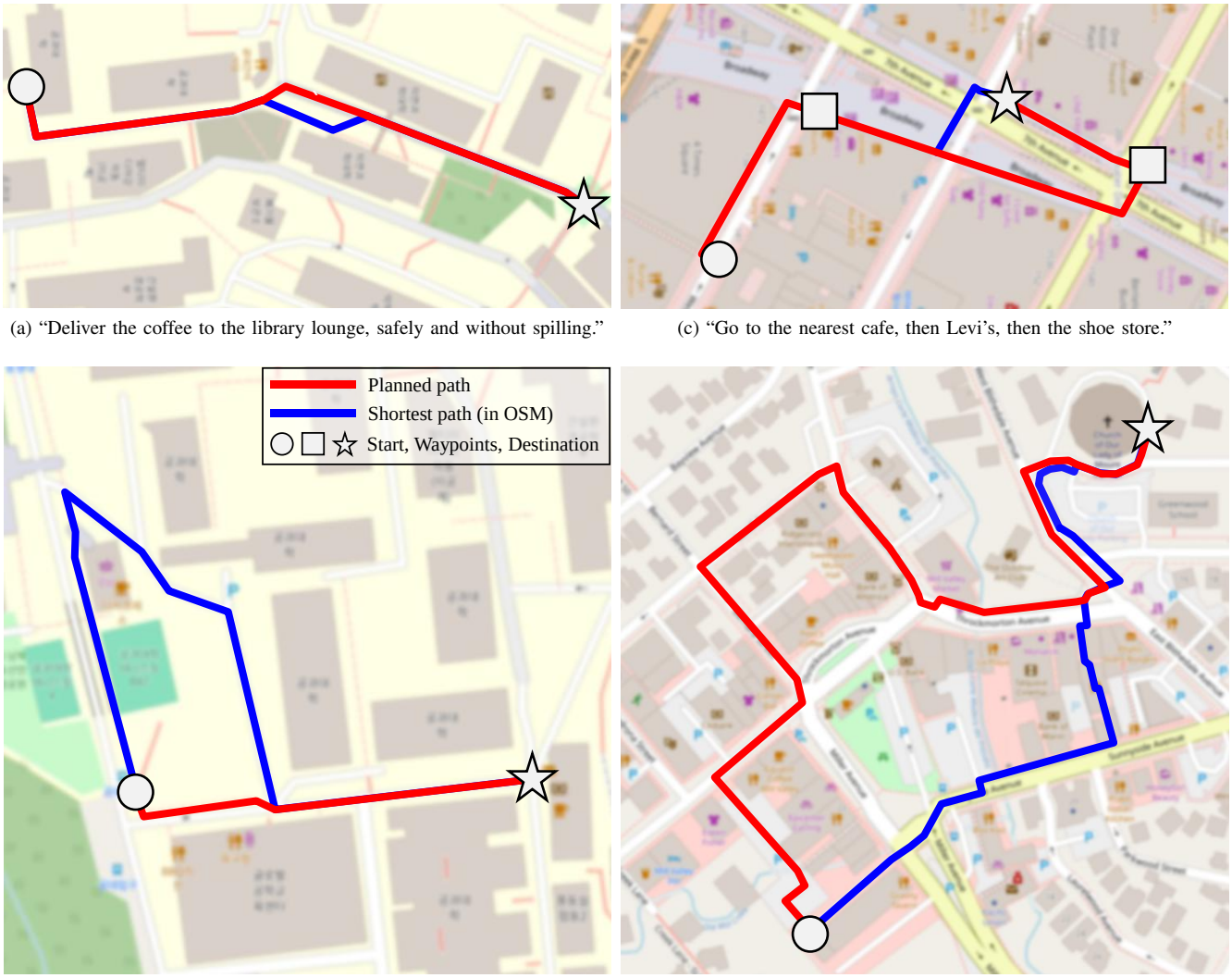
$$I = \iota(\mathcal{G}, P), \tag{9}$$

$$\Gamma = \gamma(I, T), \tag{10}$$

where I represents the image of the OSM graph path P visualized on the OSM map. ι denotes the function that generates the image I. Γ represents the feedback produced by the path evaluator, and γ denotes the function that generates Γ using I and T as inputs.

G. Navigation

The navigation module executes robot navigation by applying the graph path validated in Sec. III-F to the quadruped robot. First, for mapping and localization, LiDAR SLAM is used to generate an occupancy grid map and estimate odometry by leveraging robot odometry and LiDAR data. The coordinate alignment between the generated occupancy grid map and the OSM graph path is performed using GPS and IMU orientation information. However, if the reliability of GPS and IMU data is low due to environmental factors, the initial pose is provided by the user to perform coordinate alignment. After coordinate alignment, the navigation path for robot motion execution is generated by integrating the planner's graph route with the SLAM-derived occupancy grid map. For subgoals on the graph path that are within the occupancy grid map, A* is used to perform path planning in the free space of the occupancy grid map. For subgoals located outside the occupancy grid map, the algorithm first traces a straight line from the current position toward the



(a) "Deliver the coffee to the library lounge, safely and without spilling."

(c) "Go to the nearest cafe, then Levi's, then the shoe store."

(b) "Go to the Bank using the fastest route."

(d) "Avoid main roads and take residential streets to the church."

Fig. 5. Visualization of the generated graph paths. The left column presents results from the campus environment for (a) and (b), while the right column shows results from urban and suburban environments for (c) and (d), respectively.

target subgoal and identifies the last point that lies within free space. This point is then designated as a local goal, and a path to it is generated using the A* algorithm. From the local goal to the subgoal outside the occupancy grid map, the connection is made with a straight line. For path tracking, Model Predictive Control (MPC) [25] is employed to compute the linear and angular velocities required for the robot, based on the generated navigation path. These velocity commands are then applied to control the robot so that it follows the planned path to the destination. The overall navigation process of the robot is illustrated in Fig. 2.

IV. Experiments

A. Research Questions

The research questions addressed in BD-Nav are as follows:

Q1. Can the BD Path Planner generate routes that align with user instructions using OSM and LLMs?

Q2. Do the OSM graph features and the Path Evaluator contribute to generating accurate graph paths?

Q3. Does BD Path Planner perform robustly across diverse environments?

Q4. Can the generated OSM-based graph paths be effectively executed for real-world navigation?

B. Experimental Setup

1) Target Environments

To evaluate the performance of the BD Path Planner, one university campus and two locations each from urban and suburban areas worldwide were selected. For each environment, multiple scenarios were designed and used to conduct path planning experiments. Furthermore, the generated OSM graph paths were utilized to select two scenarios within the campus environment, which were then used to validate the performance of the BD Navigator.

2) Metrics and Data

We evaluated the performance of BD-Nav using precision, recall, and F1 score, which are commonly employed in discrete node-based graph path planning tasks, such as

TABLE II

PATH PLANNING ACCURACY BY MODEL TYPE AND DATA AVAILABILITY (%).

Case	# of Scenarios	LLM	VLM	with OSM graph features	Precision	Recall	F1
1	14	GPT-4	GPT-4o	✓	**93.15**	**96.66**	**94.87**
2		GPT-4o	GPT-4o	✓	70.23	72.29	71.25
3		Gemini-2.5-flash	GPT-4o	✓	86.48	93.98	90.07
4	11	GPT-4	-	✓	41.20	79.69	54.32
5		GPT-4	GPT-4-Turbo	✓	**85.30**	**92.72**	**88.86**
6		GPT-4	GPT-4o	✓	83.97	89.39	86.60
7	9	GPT-4	GPT-4o	-	69.87	62.16	65.79
8		GPT-4	GPT-4o	✓	**96.00**	**96.00**	**96.00**

TABLE III

PATH PLANNING ACCURACY BY ENVIRONMENT (%).

Area	# of Scenarios	Environment	Precision	Recall	F1
1	7	Urban	94.64	96.93	95.77
2	9	Urban	94.77	98.66	96.68
3	10	Suburban	89.80	91.41	90.60
4	10	Suburban	94.85	91.63	93.21

those utilizing OSM graphs. To compute these metrics, we first manually generated the ground truth graph paths corresponding to user instructions directly from OSM data. Specifically, the experimental scenarios were categorized into several types, such as "specific region and terrain selection with avoidance constraints," "shortest path navigation," and "finding the closest destination among multiple identical goal candidates." For each scenario, we examined the OSM data to design navigation commands and manually recorded the corresponding reference paths. Finally, the predicted paths were compared with the ground truth paths at the OSM node level, and each metric was calculated based on these comparisons for performance evaluation.

3) Experimental Details

First, for the path planning tests, GPT [26] and Gemini [27] models were employed to compare the performance of LLM- and VLM-based approaches. Notably, although GPT-5 is the latest version of the GPT series at the time of writing, its performance was observed to be lower than that of GPT-4 and GPT-4o when applied as the LLM and VLM within the BD Path Planner. Therefore, GPT-5 was excluded from the performance comparison experiments presented in this paper. Additionally, δ was set to 200 m, and OSM data within this radius centered at the reference point p were loaded and utilized. The temperature for both the LLM and VLM was fixed to 0 to eliminate stochasticity, ensuring reproducible experiments and reliable real-world robot operations. The MPC prediction horizon was set to 10 steps. To validate the navigation performance of the proposed BD-Nav, real-world experiments were conducted in the environments shown in Fig. 6(a). A Unitree Go1 quadruped robot was used, equipped with an Intel NUC mini-PC and a Velodyne VLP-16 LiDAR. For GPS and heading angle measurements, an Xsens MTi-680G IMU and a u-blox ANN-MB multi-band active GNSS antenna were integrated into the system. Finally, to enable real-time information exchange between the BD Path Planner and BD Navigator modules, as well as seamless integration with sensors and robot hardware, the Robot Operating System 2 (ROS2) [28] was utilized.

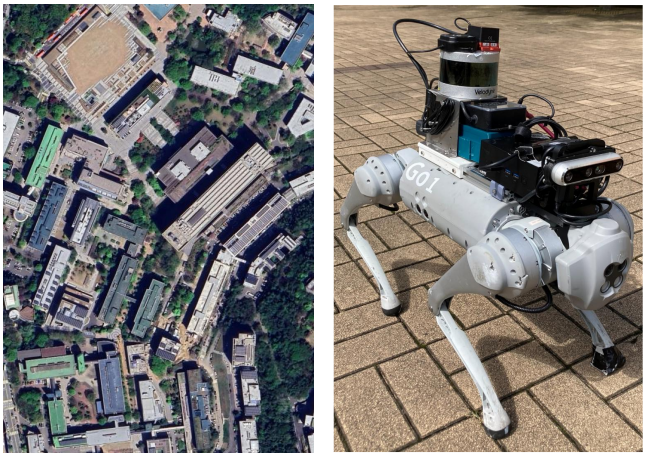
C. Evaluation

1) Path Planning

The experimental results for the BD Path Planner are presented in Fig. 5, Table II, and Table III. Fig. 5 shows four representative scenarios. Figs. 5(a) and 5(b) correspond to campus experiments, while Figs. 5(c) and 5(d) illustrate urban and suburban environments.

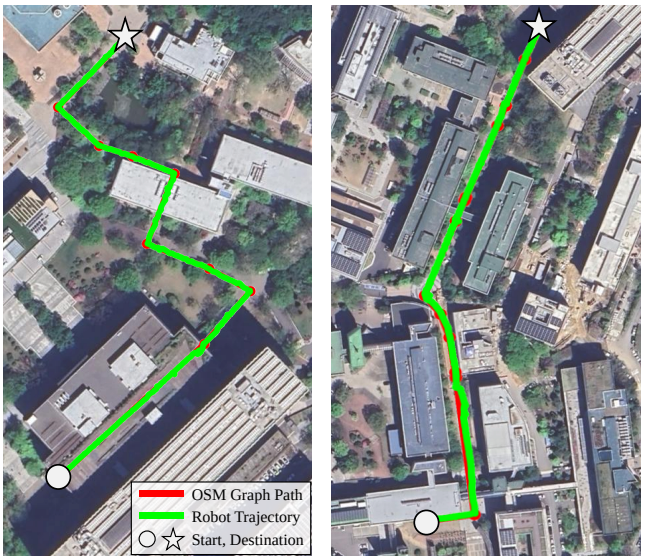
In Fig. 5(a), the BD Path Planner generates a safe route that avoids stairs and primarily follows flat terrain, even though it is longer than the shortest path, demonstrating its ability to reflect safety-related user constraints. In Fig. 5(c), the planner correctly produces a route that sequentially visits subgoals such as nearby cafes, rather than moving directly to the final destination. Similarly, in Fig. 5(d), while the shortest path follows main roads, the BD Path Planner successfully generates a route that prioritizes residential roads as instructed by the user. An interesting observation is that when the direct path to the target church was missing in the OSM data, the closest available OSM node was set as the destination, which led to an extra segment being generated beyond the church. We plan to address such cases in future work by integrating the Path Evaluator for automatic correction. Fig. 5(b) highlights the role of the Path Evaluator. While the shortest path bypasses a region due to missing OSM roads, the Path Evaluator predicts a connection between nearby nodes, shown as red points, creating a more direct route that reflects real roads and validates Q1 and Q2.

Table II compares the performance of the BD Path Planner under different configurations. GPT-4 achieved the best results among the tested LLMs and was therefore adopted in later experiments. For VLMs, adding the Path Evaluator improved performance by approximately 61.5% in terms of F1 score compared to the case without a VLM, highlighting its critical role and further validating Q2. GPT-4-Turbo performed slightly better at correcting erroneous



(a) Experimental environment [29]. (b) Robot hardware.

Fig. 6. Real-world experimental environment and robot.



(a) Scenario 1. (b) Scenario 2.

Fig. 7. Visualization of actual robot trajectories using the generated graph paths. Scenarios 1 and 2 show the results of the robot navigating along the paths generated based on the user instructions: "Bring the documents to the bank, keeping them dry." and "Deliver the coffee to the library lounge, safely and without spilling," respectively.

paths, however it was less decisive when validating correct ones, which resulted in only a small difference from GPT-4o. For efficiency, GPT-4o was used in subsequent tests. The impact of OSM graph features in the Path Weighter, described in Sec. III-D, was examined by comparing cases 7 and 8. When OSM graph features were provided, the F1 score improved by 45.92%. Without this information, the LLM relied on general language knowledge and sometimes produced incorrect weight functions. For example, when given the instruction "avoid slopes," the correct OSM key should be `incline`. Without explicit feature data, the LLM may generate a non-functional key such as `slope`, which leads to failure in applying the intended constraint during graph search.

Finally, Table III evaluates the BD Path Planner across four diverse urban and suburban regions, covering a total of 36 scenarios. Using the best configuration consisting of GPT-4 as the LLM, GPT-4o as the VLM, and the inclusion of OSM graph features, the planner achieved F1 scores above 90% in all regions. These results demonstrate the robustness and generalizability of the proposed approach across real-world environments and confirm Q3.

2) Navigation

We conducted real-world navigation experiments using a quadruped robot to validate Q4 presented in Sec. IV-A. The experimental procedure was as follows. First, the robot was positioned at the starting location, and its initial coordinates were obtained via GPS. A user instruction was then provided, and the BD Path Planner module generated an OSM-based graph path according to the instruction. The generated graph path was subsequently passed to the BD Navigator module, which applied SLAM, navigation path planning, and control algorithms to autonomously drive the robot along the path toward the target destination. The resulting trajectories of the robot are shown in Fig. 7. Two representative scenarios were evaluated in this experiment.

The first scenario was based on the following instruction: "Deliver this coffee to my friend in the lounge at the central library. You may go slowly, but please avoid dangerous areas such as stairs, slopes, or construction zones, and take the safest route possible." The generated path and the actual robot trajectory are illustrated in Fig. 7(a). This path was approximately 330 m long and included rotations along the vertical axis. As shown in Fig. 5(a), the shortest path between the start and the destination would have included a staircase in the middle. However, because the instruction explicitly required the robot to avoid hazardous areas such as stairs, the robot selected a safer, mostly flat path instead. Although this resulted in a slightly longer overall distance, the robot was able to navigate stably. During the run, the robot occasionally deviated slightly from a straight line, which can be attributed to the real-time SLAM process used to build the occupancy grid map for path planning. Temporary misclassification of the floor as occupied pixels due to the robot's body shaking, or the presence of obstacles such as parked bicycles and motorcycles, occasionally generated detour paths. These detours caused the robot to slightly deviate from the optimal trajectory.

The second scenario was based on the following instruction: "Deliver this document to the staff at the bank. It is acceptable to take a longer route, but please prioritize paths with overhead shelter to avoid the rain." The path generated for this scenario and the corresponding trajectory are shown in Fig. 7(b). This path was approximately 360 m long, requiring the robot to pass through two building tunnels and make four 90-degree turns. The inclusion of building tunnels in the path reflects the instruction's request to prioritize sheltered routes, even at the cost of taking a longer path, in order to avoid exposure to rain. Compared to the first scenario, this route included more smooth, marble-paved

segments, which reduced the robot's body shaking and led to fewer temporary errors in the generated occupancy grid map. As a result, the robot successfully navigated along the OSM graph path and reached the final destination, the bank, in a stable and reliable manner.

V. Conclusion

In this paper, we proposed BD-Nav, a framework composed of two main components: the BD Path Planner, which uses LLM- and VLM-based modules to integrate user instructions with OSM data and generate routes that include destinations and intermediate paths, and the BD Navigator, which executes these routes on a quadruped robot in real-world environments. The LLM interprets user instructions and generates weight function code for OSM-based path planning, while the VLM verifies the generated route. Validated routes are then executed using rule-based navigation. To enable deployment in diverse environments, we did not use a pre-built occupancy grid map. Instead, we employed real-time SLAM with LiDAR data for localization and occupancy grid map generation. BD-Nav was validated through real-world experiments using OSM data, demonstrating its practicality.

In future work, we will enhance the Path Evaluator of BD-Nav so that it can dynamically update graph paths in response to user instructions and real-world conditions. We also plan to develop an active navigator that leverages perception and exploration to identify safe and feasible alternatives whenever OSM data disagree with actually traversable routes. Furthermore, to directly translate natural-language instructions into path constraints and costs, we aim to construct and exploit an ontology that links the OSM tag schema with shared culture, common sense, and traffic regulations. These advances will enable robots to interpret language-based instructions with higher accuracy, ultimately improving navigation success rates for real-world environments.

References

[1] M. Misaros, O.-P. Stan, I.-C. Donca, and L.-C. Miclea, "Autonomous robots for services—state of the art, challenges, and research areas," *Sensors*, vol. 23, no. 10, p. 4962, 2023.

[2] D. Chen and R. Mooney, "Learning to interpret natural language navigation instructions from observations," in *Proceedings of the AAAI Conference on Artificial Intelligence*, vol. 25, no. 1, 2011, pp. 859–865.

[3] J. Guerreiro, D. Sato, S. Asakawa, H. Dong, K. M. Kitani, and C. Asakawa, "Cabot: Designing and evaluating an autonomous navigation robot for blind people," in *Proceedings of the 21st international ACM SIGACCESS conference on computers and accessibility*, 2019, pp. 68–82.

[4] S.-W. Kim, G.-P. Gwon, W.-S. Hur, D. Hyeon, D.-Y. Kim, S.-H. Kim, D.-K. Kye, S.-H. Lee, S. Lee, M.-O. Shin *et al.*, "Autonomous campus mobility services using driverless taxi," *IEEE Transactions on intelligent transportation systems*, vol. 18, no. 12, pp. 3513–3526, 2017.

[5] T. Kollar, S. Tellex, D. Roy, and N. Roy, "Toward understanding natural language directions," in *2010 5th ACM/IEEE International Conference on Human-Robot Interaction (HRI)*. IEEE, 2010, pp. 259–266.

[6] P. Forte, H. Gupta, H. Andreasson, U. Köckemann, and A. J. Lilienthal, "On robust context-aware navigation for autonomous ground vehicles," *IEEE Robotics and Automation Letters*, 2024.

[7] S. Woo and S.-W. Kim, "Context-based visual-language place recognition," *arXiv preprint arXiv:2410.19341*, 2024.

[8] D. Jung, K. Kim, and S.-W. Kim, "Gotpr: General outdoor text-based place recognition using scene graph retrieval with openstreetmap," *IEEE Robotics and Automation Letters*, 2025.

[9] J. Bao and R. Yonetani, "Path planning using instruction-guided probabilistic roadmaps," *arXiv preprint arXiv:2502.16515*, 2025.

[10] S. C. Marcelyn, Y. Gao, Y. Zhang, X. Gao, and G. Chen, "Pathgpt: Leveraging large language models for personalized route generation," *arXiv preprint arXiv:2504.05846*, 2025.

[11] M. Kuribayashi, K. Uehara, A. Wang, S. Morishima, and C. Asakawa, "Wanderguide: Indoor map-less robotic guide for exploration by blind people," in *Proceedings of the 2025 CHI Conference on Human Factors in Computing Systems*, 2025, pp. 1–21.

[12] C. Kim, K. Kim, M. Oh, H. Baek, J. Lee, D. Jung, S. Woo, Y. Woo, J. Tucker, R. Firoozi *et al.*, "E2map: Experience-and-emotion map for self-reflective robot navigation with language models," *arXiv preprint arXiv:2409.10027*, 2024.

[13] D. Shah, B. Osiński, S. Levine *et al.*, "Lm-nav: Robotic navigation with large pre-trained models of language, vision, and action," in *Conference on robot learning*. PMLR, 2023, pp. 492–504.

[14] J. Wang, D. Huo, Z. Xu, Y. Shi, Y. Yan, Y. Wang, C. Gao, Y. Qiao, and G. Zhou, "Openbench: A new benchmark and baseline for semantic navigation in smart logistics," *arXiv preprint arXiv:2502.09238*, 2025.

[15] C. Huang, O. Mees, A. Zeng, and W. Burgard, "Visual language maps for robot navigation," *arXiv preprint arXiv:2210.05714*, 2022.

[16] M. Haklay and P. Weber, "Openstreetmap: User-generated street maps," *IEEE Pervasive computing*, vol. 7, no. 4, pp. 12–18, 2008.

[17] W. Burgard, A. B. Cremers, D. Fox, D. Hähnel, G. Lakemeyer, D. Schulz, W. Steiner, and S. Thrun, "Experiences with an interactive museum tour-guide robot," *Artificial intelligence*, vol. 114, no. 1-2, pp. 3–55, 1999.

[18] S. Macenski, F. Martín, R. White, and J. G. Clavero, "The marathon 2: A navigation system," in *2020 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*. IEEE, 2020, pp. 2718–2725.

[19] D. Shah, B. Eysenbach, G. Kahn, N. Rhinehart, and S. Levine, "Ving: Learning open-world navigation with visual goals," in *2021 IEEE International Conference on Robotics and Automation (ICRA)*. IEEE, 2021, pp. 13 215–13 222.

[20] S. Liu, A. Hasan, K. Hong, R. Wang, P. Chang, Z. Mizrachi, J. Lin, D. L. McPherson, W. A. Rogers, and K. Driggs-Campbell, "Dragon: A dialogue-based robot for assistive navigation with visual language grounding," *IEEE Robotics and Automation Letters*, vol. 9, no. 4, pp. 3712–3719, 2024.

[21] E. W. Dijkstra, "A note on two problems in connexion with graphs," in *Edsger Wybe Dijkstra: his life, work, and legacy*, 2022, pp. 287–290.

[22] A. Radford, J. W. Kim, C. Hallacy, A. Ramesh, G. Goh, S. Agarwal, G. Sastry, A. Askell, P. Mishkin, J. Clark *et al.*, "Learning transferable visual models from natural language supervision," in *International conference on machine learning*. PMLR, 2021, pp. 8748–8763.

[23] P. E. Hart, N. J. Nilsson, and B. Raphael, "A formal basis for the heuristic determination of minimum cost paths," *IEEE transactions on Systems Science and Cybernetics*, vol. 4, no. 2, pp. 100–107, 1968.

[24] R. Manvi, S. Khanna, G. Mai, M. Burke, D. Lobell, and S. Ermon, "Geollm: Extracting geospatial knowledge from large language models," *arXiv preprint arXiv:2310.06213*, 2023.

[25] C. E. Garcia, D. M. Prett, and M. Morari, "Model predictive control: Theory and practice—a survey," *Automatica*, vol. 25, no. 3, pp. 335–348, 1989.

[26] J. Achiam, S. Adler, S. Agarwal, L. Ahmad, I. Akkaya, F. L. Aleman, D. Almeida, J. Altenschmidt, S. Altman, S. Anadkat *et al.*, "Gpt-4 technical report," *arXiv preprint arXiv:2303.08774*, 2023.

[27] G. Comanici, E. Bieber, M. Schaekermann, I. Pasupat, N. Sachdeva, I. Dhillon, M. Blistein, O. Ram, D. Zhang, E. Rosen *et al.*, "Gemini 2.5: Pushing the frontier with advanced reasoning, multimodality, long context, and next generation agentic capabilities," *arXiv preprint arXiv:2507.06261*, 2025.

[28] S. Macenski, T. Foote, B. Gerkey, C. Lalancette, and W. Woodall, "Robot operating system 2: Design, architecture, and uses in the wild," *Science Robotics*, vol. 7, no. 66, p. eabm6074, 2022.

[29] Google Earth, "Google Earth Imagery, Maxar Technologies, CNES / Airbus, Landsat / Copernicus, USDA, USGS," Google Earth, 2025, accessed September 12, 2025.